TP Morpion - Rapport

Groupe : Ahmed Adel BEREKSI REGUIG, Ismail ELBOUCH

Reproduction des résultats

Le code est en Python 3 et n'utilise que la bibliothèque standard.

- tictactoe.py : la classe TicTacToe
- rl.py : les adversaires, la value function, greedy_move, play_game et play_game2
- main.py : les expériences

Commandes :

- python main.py : lance tout (exercice 2, exercice 3, self-play, symétries), environ 15 secondes
- python main.py 2 : seulement l'exercice 2
- python main.py 3 : entraînement contre opponent_random et opponent_next, sauvegarde value_random.pkl et value_next.pkl
- python main.py selfplay : deux IA qui apprennent l'une contre l'autre
- python main.py sym : comparaison avec et sans symétries
- python main.py jouer : jouer contre l'IA (à lancer après python main.py 3)

Notre IA joue toujours X (elle commence). Pour évaluer une value function on joue 2000 à 5000 parties en mode totalement greedy (eps = 0) et sans apprentissage. Il n'y a pas de graine fixée, les pourcentages varient donc d'un ou deux points d'un lancement à l'autre.

Exercice 1 - Une classe TicTacToe

1. La grille est une liste de 9 cases qui contiennent " ", "X" ou "O". L'attribut player indique qui doit jouer, X au départ.

2. play(i) lève une ValueError si la case est déjà prise, sinon pose le symbole et passe au joueur suivant.

3. __str__ affiche la grille sur 3 lignes avec des séparateurs.

4. has_winner teste les 8 alignements (3 lignes, 3 colonnes, 2 diagonales). On a aussi une méthode winner qui renvoie "X", "O" ou None.

5. is_draw renvoie True si les 9 cases sont remplies et qu'il n'y a pas de gagnant.

6. reset remet la grille vide et redonne la main à X.

7. undo(i) vide la case i et redonne la main au joueur qui avait joué ce coup.

8. allowed_moves est une propriété qui renvoie la liste des cases vides.

Exercice 2 - Jouer !

1. opponent_random renvoie random.choice(game.allowed_moves).

2. opponent_next essaye chaque coup possible avec play puis undo. S'il y a un coup gagnant il le joue, sinon il joue au hasard.

3. Sur 10000 parties, random (X) contre next (O) : X 39%, O 52%, nuls 8%. Next (X) contre random (O) : X 81%, O 12%, nuls 7%. Le joueur next est donc bien plus fort que le joueur aléatoire.

4. La value function est un defaultdict(lambda: 0.5), donc un état jamais vu vaut 0.5.

5. __hash__ renvoie un entier unique pour chaque grille : la grille est vue comme un nombre en base 3 (case vide = 0, X = 1, O = 2). Pour aller plus vite cet entier est mis à jour directement dans play et undo au lieu d'être recalculé. On a aussi défini __eq__. Comme l'objet TicTacToe change au cours de la partie, on indexe le dictionnaire avec hash(game) et pas avec l'objet lui-même, sinon la clé changerait après avoir été insérée.

6. greedy_move essaye chaque coup possible (play, lecture de la valeur de l'état obtenu, undo) et choisit au hasard parmi les coups qui ont la valeur maximale.

7. play_game(V, opponent) joue une partie complète : notre IA joue avec greedy_move, l'adversaire avec la fonction donnée. Elle renvoie "X", "O" ou "nul".

8. Sans apprentissage toutes les valeurs restent à 0.5, donc greedy_move joue au hasard. Sur 20000 parties contre opponent_random on obtient X 58.3%, O 29.0%, nuls 12.7%, ce qui correspond aux 59%, 29% et 12% attendus.

Exercice 3 - Temporal-Difference

1. Quand la partie est finie on met la valeur de l'état final à 1 si notre IA a gagné, 0 si elle a perdu ou fait match nul.

2. Après chaque coup de notre IA on applique la formule du cours : V(s) = V(s) + alpha * (V(s') - V(s)), où s est l'état après notre coup précédent et s' l'état après notre coup actuel. alpha = 0.1. Quand l'adversaire gagne, l'état après notre dernier coup est mis à jour vers 0.

3. Question : si on joue uniquement de manière greedy, l'IA ne visite que les états qu'elle pense déjà bons. Dès qu'un coup lui a donné une victoire elle va le rejouer et ne testera jamais les autres, même s'ils sont meilleurs. Elle ne joue pas forcément moins bien : dans notre test (50000 parties en pur greedy contre random) elle gagne quand même environ 98% du temps, parce qu'un joueur aléatoire est facile à battre. Mais elle n'est pas forcément meilleure non plus, rien ne garantit qu'elle trouve la stratégie optimale, ça dépend beaucoup des premières parties. Une valeur initiale plus élevée (par exemple 1) règle en partie le problème : les états jamais visités ont l'air meilleurs que ceux déjà testés dont la valeur a baissé, donc l'IA est poussée à les essayer. C'est le même principe que l'initialisation optimiste du TP bandit.

4. Avec une probabilité de 5% notre IA joue un coup au hasard au lieu du coup greedy (paramètre eps de play_game). Résultats en mode greedy après entraînement :

Entraînement	Parties	X gagne	O gagne	Nuls
contre random	10000	98.3%	0.8%	0.9%
contre random	50000	98.5%	0.1%	1.4%
contre next	10000	97.0%	0.5%	2.5%
contre next	50000	99.8%	0.2%	0.0%

Au départ (sans apprentissage) on avait 58% de victoires contre random, l'apprentissage marche donc très bien. La value function contient environ 2300 états à la fin.

5. Question : si on ne met pas à jour la value function après un coup exploratoire (paramètre learn_explore=False), elle estime la probabilité de gagner en jouant ensuite de manière greedy, c'est-à-dire avec la meilleure stratégie trouvée. Si on apprend aussi pendant l'exploration, elle estime la probabilité de gagner avec la stratégie qu'on suit vraiment, exploration comprise. Les valeurs sont alors un peu plus basses près des états où un mauvais coup coûte cher. La première version est

meilleure si on arrête d'explorer après l'entraînement (on joue alors vraiment greedy), la deuxième est plus prudente pendant l'entraînement. Dans nos tests contre random les deux donnent à peu près le même résultat (98.9% de victoires sans mise à jour pendant l'exploration).

6. La value function est sauvegardée avec pickle dans value_random.pkl et value_next.pkl, et python main.py jouer recharge value_next.pkl pour jouer contre un humain.

7. Question : si les deux joueurs utilisent la même méthode d'apprentissage, l'adversaire n'est plus fixe : chacun s'adapte aux faiblesses de l'autre, qui les corrige à son tour. Les deux devraient converger vers un jeu optimal, et au morpion un jeu optimal des deux côtés donne toujours match nul. La value function apprise est donc différente : contre un joueur aléatoire l'IA apprend à profiter de ses erreurs, en self-play elle apprend surtout à ne jamais perdre.

8. play_game2(V1, V2) fait jouer deux IA qui ont chacune leur value function (V1 pour X, V2 pour O). Chacune évalue les états après ses propres coups et fait sa mise à jour TD. Premier essai avec un nul valant 0 comme une défaite : X gagnait 100% des parties, parce que O n'avait aucun intérêt à bloquer (perdre ou faire nul lui donnait la même valeur). En donnant la valeur 0.5 au match nul on obtient bien le résultat attendu :

Parties de self-play	X gagne	O gagne	Nuls
20000	0%	0%	100%
100000	0%	0%	100%

La value function de X apprise en self-play gagne quand même 99% des parties contre random et 98.7% contre next, sans jamais perdre.

9. Question : beaucoup de grilles sont identiques à une rotation ou une symétrie près (8 transformations : 4 rotations, avec ou sans miroir). On peut ramener chaque grille à une forme canonique, par exemple la plus petite des 8 en base 3, et utiliser cette forme comme clé de la value function. On l'a implémenté (option sym=True, fonction key dans rl.py). On passe d'environ 2150 à environ 410 états pour 20000 parties, et chaque partie met à jour la valeur de toutes les grilles symétriques d'un coup, donc l'apprentissage est plus rapide en nombre de parties : 99.8% de victoires après 20000 parties contre 98.5% sans symétries.

Par contre, si l'adversaire n'est pas symétrique (s'il préfère par exemple le côté droit), deux grilles symétriques n'ont pas la même probabilité de gain. Les fusionner ferait perdre de l'information et l'IA ne pourrait pas exploiter cette préférence. Dans ce cas les états symétriques ne devraient pas avoir la même valeur.

10. Améliorations proposées et implémentées :

- hash incrémental : la clé en base 3 est mise à jour dans play et undo, on évite de reconstruire une chaîne de caractères à chaque appel (ce qui serait le cas en passant par __str__) ;
- symétries (question 9) : moins de parties nécessaires, mais chaque coup coûte plus cher car on calcule les 8 transformations (2.1 s contre 0.4 s pour 20000 parties), c'est donc plus intéressant quand une partie coûte cher ;
- les états finaux reçoivent directement leur valeur 0 ou 1, donc greedy_move voit tout de suite un coup gagnant dès qu'il a été rencontré une fois.

Autres pistes non implémentées : faire diminuer alpha et eps au fil des parties (le cours indique qu'avec un alpha décroissant on converge vers les vraies probabilités), commencer l'entraînement contre opponent_next puis continuer en self-play, ou initialiser les états de façon optimiste pour explorer plus vite.